MOBILE APPLICATION AND DEVELOPMENT

App Architecture: MVC, MVP, MVVM & Room

Keep your code tidy, your data safe and your app smooth

Ekramul Haque Amin · Lecturer, AI & Data Science · Dhaka International University

TODAY'S ROADMAP

What We Will Cover

1. Why big apps turn into spaghetti, and how architecture saves us
2. Three patterns: MVC, then MVP, then MVVM
3. Room: your own database inside the app
4. The Repository: one door to all your data
5. A Contact Manager built from all of it, plus shimmer loading

DID YOU KNOW

A MainActivity can quietly grow to hundreds of lines

Screen code, data code and business rules all pile into one file.

SPOT THE MESS

The Activity That Does Everything

`MainActivity.java`

findViewById · button clicks · email validation · SQL queries · JSON parsing · date formatting · navigation · error messages · saving files · business rules

> One class is the screen, the brain and the storage room, all at once.

WHAT GOES WRONG

Four Signs of Spaghetti Code

Hard to read

Scroll for minutes to find one rule

Hard to test

You must run the whole app to check one line

Hard to change

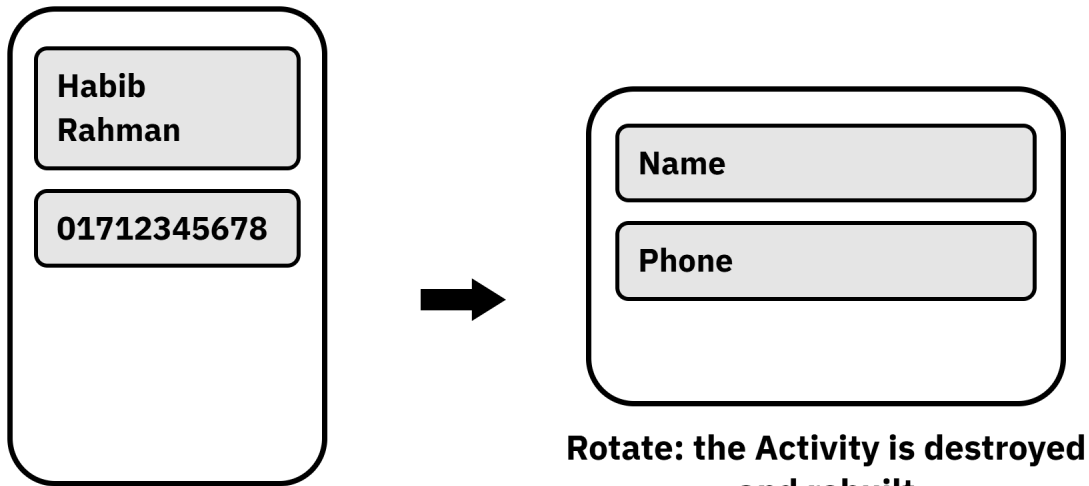
Fix one thing, break three others

Hard to share

Two teammates edit the same giant file

A CLASSIC SURPRISE

Rotate the Phone, Lose Your Data



User types a name and a phone number

Rotate: the Activity is destroyed and rebuilt

> Anything stored in the Activity’s variables disappears. Architecture gives data a safer home.

THE BIG IDEA

Architecture = Who Is Responsible for What

Architecture is not a library. It is an agreement about where each kind of code lives, so every class has one job and every teammate knows where to look.

One job each

A class that draws the screen does not also talk to the database

Easy to find

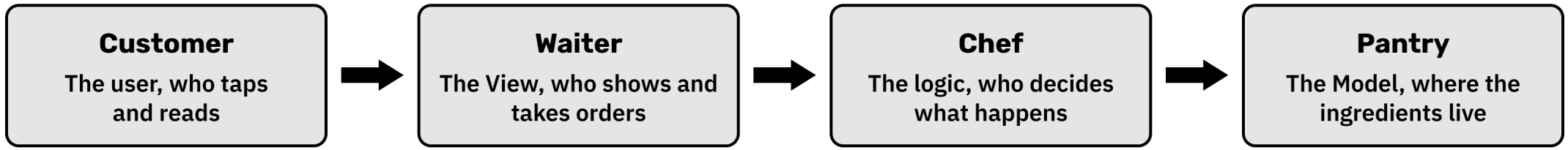
Bug in the data? Look in the data layer

Easy to change

Swap the database without touching the screen

AN ANALOGY

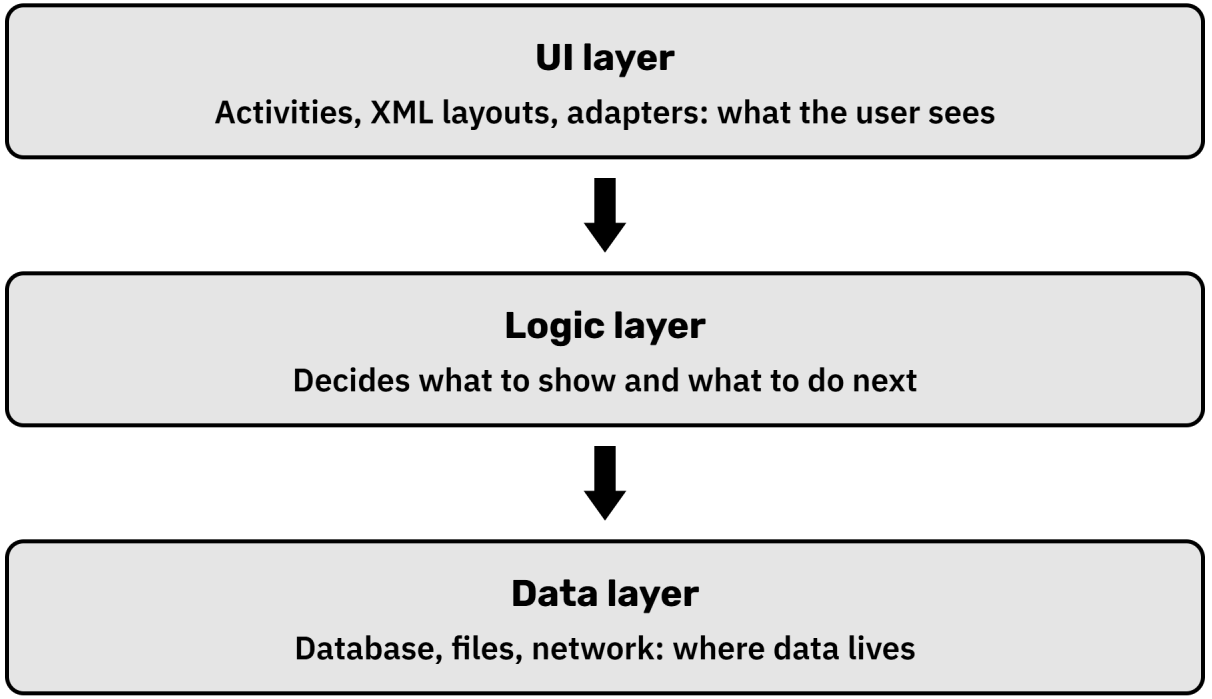
Think of a Restaurant



> The customer never walks into the pantry. The chef never serves tables. Everyone sticks to their job.

THE SAME IDEA IN CODE

Three Layers



QUICK POLL

Where would you put the code that checks the password is at least six characters?

- A. Inside the Activity
- B. Inside the XML layout
- C. In its own logic class
- D. In the database

> Keep your answer. We will come back to it at the end.

QUICK RECAP

Why Architecture

- Huge Activities are hard to read, test and change
- Rotating the screen can wipe data stored in the Activity
- Architecture means giving every class one clear job
- Three layers: UI, logic and data

> Next up: the oldest pattern, MVC.

PATTERN 1

MVC

Model · View · Controller

Fun fact: MVC was described at Xerox PARC in the late 1970s, long before smartphones existed.

THREE ROLES

What M, V and C Mean

Model

The data and its rules, such as a Contact and a list of contacts

View

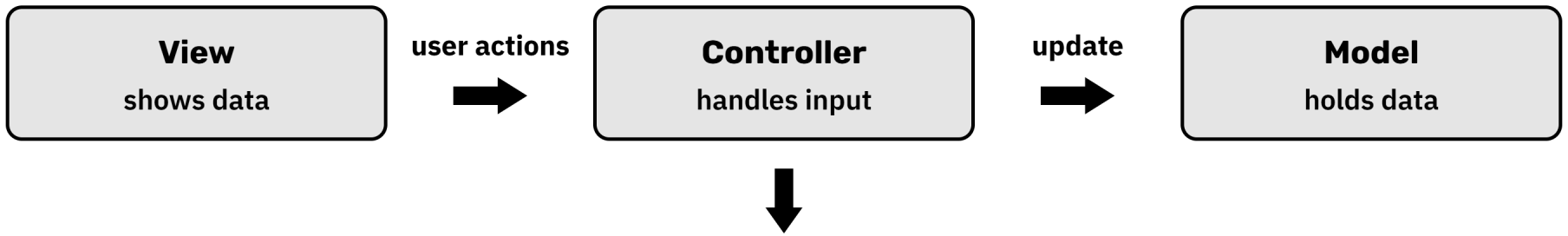
What the user sees: layouts, buttons and lists

Controller

Receives the user’s actions, updates the Model and refreshes the View

WHO TALKS TO WHOM

The MVC Flow



> In classic MVC, the Model can notify the View directly when its data changes.

SEE IT ON A SCREEN

MVC on an Add Contact Screen



- View: this screen and its XML
- Controller: the Save button’s click code
- Model: the Contact and the contact list

MVC IN CODE

The Model

```
public class Contact {
    public String name;
    public String phone;
    public Contact(String name, String phone) {
        this.name = name;
        this.phone = phone;
    }
}

public class ContactStore {
    private final List<Contact> list = new ArrayList<>();
    public void add(Contact c) { list.add(c); }
    public List<Contact> getAll() { return list; }
}
```

MVC IN CODE

The Controller (in the Activity)

```
public class MainActivity extends AppCompatActivity {
    ContactStore store = new ContactStore();       // Model

    protected void onCreate(Bundle b) {
        super.onCreate(b);
        setContentView(R.layout.activity_main);    // View

        btnSave.setOnClickListener(v → {           // Controller
            store.add(new Contact(etName.getText().toString(),
                                  etPhone.getText().toString()));
            adapter.notifyDataSetChanged();
        });
    }
}
```

THE ANDROID TWIST

The Activity Plays Two Roles



> Android Activities touch the screen and also handle logic, so View and Controller blur together and the Activity grows again.

A FAIR VERDICT

MVC: Good and Not So Good

Good

Simple to understand. Separates data from screen. A fine starting point

Not so good

The Activity ends up as both View and Controller. Logic is hard to test without a device

QUICK RECAP

MVC

- Model holds data, View shows it, Controller reacts to the user
- Easy to learn, and the first step away from spaghetti
- In Android the Activity mixes View and Controller

> Next up: MVP, where the Activity finally stops thinking.

PATTERN 2

MVP

Model · View · Presenter

In MVP the Activity becomes a thin View: it displays data and passes on taps, nothing more.

A CLEANER SPLIT

The Dumb View, The Smart Presenter

View

The Activity. It only displays and forwards events. It has no decisions to make

Presenter

The brain. It checks input, talks to the Model and tells the View what to show

Model

Data and storage, exactly as before

WHO TALKS TO WHOM

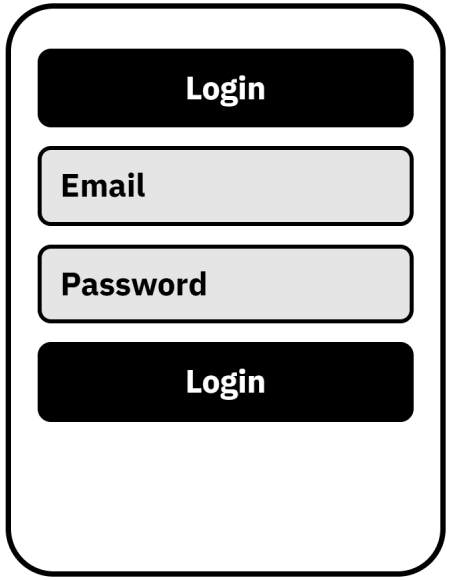
The MVP Flow



> The View and the Model never talk directly. Every conversation goes through the Presenter.

AN EXAMPLE

A Login Screen with MVP



- Tap Login: the View calls presenter.onLoginClicked()
- Presenter checks the email and the password
- Wrong? Presenter tells the View to showError()
- Right? Presenter tells the View to goHome()

MVP IN CODE

The Contract

```
public interface LoginContract {

    interface View {
        void showError(String message);
        void goHome();
    }

    interface Presenter {
        void onLoginClicked(String email, String password);
    }
}
```

> **A contract is an interface that says what each side can ask of the other.**

MVP IN CODE

The Presenter

```
public class LoginPresenter implements LoginContract.Presenter {
    private final LoginContract.View view;

    public LoginPresenter(LoginContract.View view) { this.view = view; }

    @Override
    public void onLoginClicked(String email, String password) {
        if (email.isEmpty()) { view.showError("Email is required"); return; }
        if (password.length() < 6) { view.showError("At least 6 characters"); return; }
        view.goHome();
    }
}
```

> **No Android classes in sight, so this is plain Java.**

MVP IN CODE

The View (the Activity)

```
public class LoginActivity extends AppCompatActivity
        implements LoginContract.View {

    private LoginContract.Presenter presenter;

    protected void onCreate(Bundle b) {
        super.onCreate(b);
        setContentView(R.layout.activity_login);
        presenter = new LoginPresenter(this);

        btnLogin.setOnClickListener(v → presenter.onLoginClicked(
            etEmail.getText().toString(), etPassword.getText().toString()));
    }

    public void showError(String m) { Toast.makeText(this, m, Toast.LENGTH_SHORT).show(); }
    public void goHome() { startActivity(new Intent(this, HomeActivity.class)); }
}
```

THE BIG WIN

Test Without a Phone

Plain Java

The Presenter has no Android code, so it runs on your laptop

Instant tests

A unit test runs in milliseconds, with no emulator

Fake the View

Pass a pretend View and check that showError() was called

> **Business rules you can test in a blink are rules you will trust.**

SIDE BY SIDE

MVC vs. MVP

	MVC	MVP
Who handles input	Controller, often inside the Activity	Presenter, a separate class
Activity’s job	View and Controller together	View only
View talks to Model?	Sometimes	Never
Testing	Harder	Easier

A FAIR VERDICT

MVP: Good and Not So Good

Good

Dumb Views, testable logic, clear contracts

Not so good

Lots of interfaces. The Presenter must hold the View, and rotation can leave it pointing at a dead Activity

QUICK RECAP

MVP

- The View only displays and forwards taps
- The Presenter holds the logic and talks to the Model
- A contract interface connects them and makes testing easy

> Next up: MVVM, the pattern Google recommends.

PATTERN 3

MVVM

Model · View · ViewModel

In MVVM the View observes the ViewModel’s data and updates itself whenever it changes.

THE MODERN PATTERN

What M, V and VM Mean

Model

Data: the database, files and network

View

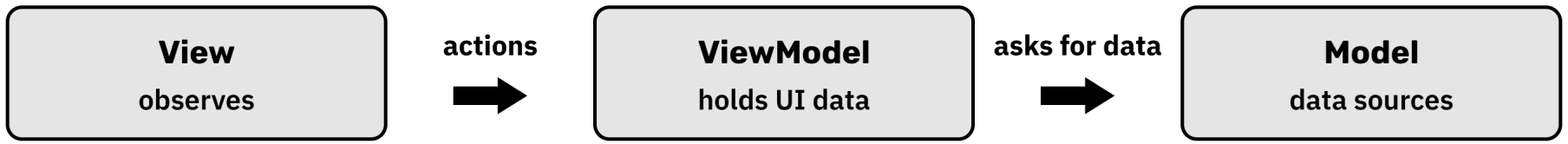
Activity and XML. It observes the ViewModel

ViewModel

Holds the screen’s data and survives rotation. It never knows about the View

WHO TALKS TO WHOM

The MVVM Flow



> The ViewModel never holds a reference to the View. The View watches it from the outside.

AN ANALOGY

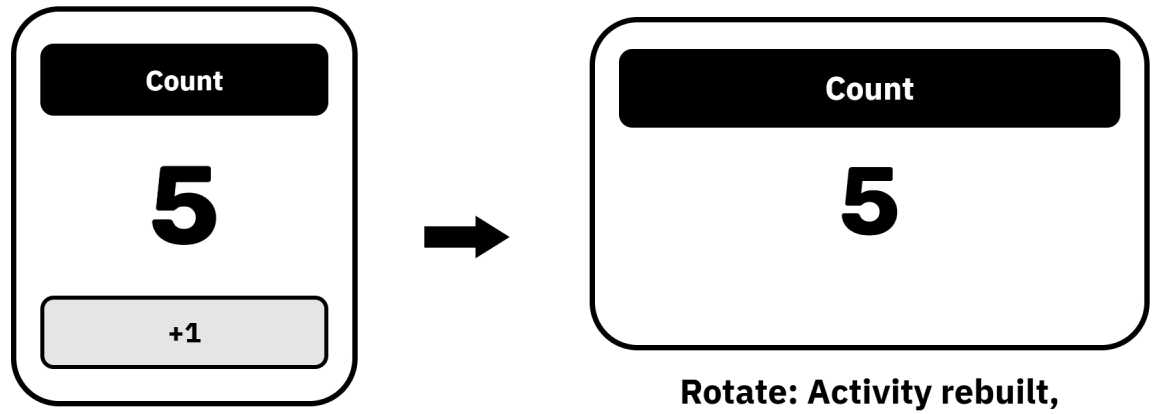
Subscribe and Get Notified



> You do not keep refreshing the channel. When something new appears, you are told. That is observing.

THE SUPERPOWER

The ViewModel Survives Rotation



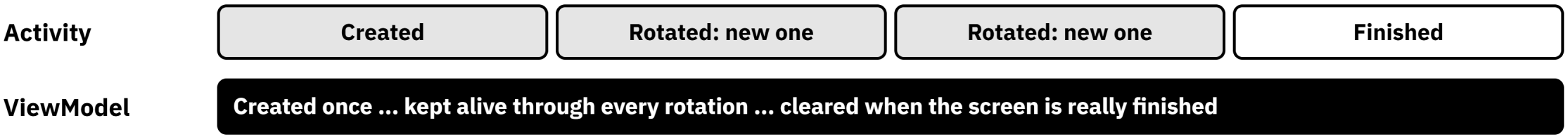
Count is 5

Rotate: Activity rebuilt, ViewModel kept

> Same data, new screen. No saving and restoring by hand.

WHY IT WORKS

Two Different Lifetimes



> The ViewModel lives longer than any single Activity instance.

THE WATCHING PART

What Is LiveData?

Observable

The View registers an observer and gets called when the value changes

Lifecycle-aware

Updates only reach screens that are visible, so no crashes

Always fresh

A new observer instantly receives the latest value

MVVM IN CODE

The ViewModel

```
public class CounterViewModel extends ViewModel {

    private final MutableLiveData<Integer> count = new MutableLiveData<>(0);

    public LiveData<Integer> getCount() { return count; }

    public void increment() {
        count.setValue(count.getValue() + 1);
    }
}
```

> **Expose LiveData, keep MutableLiveData private. Only the ViewModel may change the value.**

MVVM IN CODE

The View Observes

```
CounterViewModel vm =
        new ViewModelProvider(this).get(CounterViewModel.class);

vm.getCount().observe(this, value →
        txtCount.setText(String.valueOf(value)));

btnPlus.setOnClickListener(v → vm.increment());
```

> **ViewModelProvider hands back the same ViewModel after a rotation. That is the magic.**

EVEN TIDIER

MVVM with ViewBinding

```
private ActivityMainBinding binding;

protected void onCreate(Bundle b) {
    super.onCreate(b);
    binding = ActivityMainBinding.inflate(getLayoutInflater());
    setContentView(binding.getRoot());

    vm.getCount().observe(this, v →
            binding.txtCount.setText(String.valueOf(v)));

    binding.btnPlus.setOnClickListener(v → vm.increment());
}
```

> **ViewBinding replaces findViewById, so views are typed and typos fail at compile time.**

FOLLOW ONE TAP

What Happens When You Press +

1. The user taps the + button
2. The Activity calls vm.increment()
3. The ViewModel updates its LiveData
4. The observer in the Activity fires
5. The TextView shows the new number

ALL THREE TOGETHER

MVC vs. MVP vs. MVVM

	MVC	MVP	MVVM
Middle layer	Controller	Presenter	ViewModel
Knows the View?	Yes	Yes, via an interface	No
UI updates by	Controller calls the View	Presenter calls the View	View observes data
Survives rotation	No	No	Yes
Testing	Harder	Easy	Easy

CHOOSING ONE

Which Pattern Should You Use?

Learning or tiny app

MVC is fine. Just keep the Activity tidy

Heavy unit-testing

MVP is a solid choice

Modern Android app

MVVM. Google recommends it, and Jetpack libraries are built for it

> **This course's project template: MVVM with Room.**

QUICK RECAP

MVVM

- ViewModel holds the screen’s data and survives rotation
- LiveData lets the View observe changes
- The ViewModel never references the View
- Google recommends MVVM for Android

> Next up: Room, the database MVVM loves to work with.

TOPIC 4

Room

Your own database inside the app

Contacts, notes, to-dos: the data should still be there after the app closes.

WHY ROOM?

SQLite Without the Pain

Less boilerplate

Write an interface and an annotation instead of pages of cursor code

Checked at compile time

A typo in your SQL is an error before you run the app

Works with LiveData

Queries can update your screen automatically

THE THREE PIECES

Entity, DAO, Database



> Think of it as: the table, the menu of actions, and the building they live in.

SET UP

Add Room to Gradle

```
dependencies {
    def room_version = "2.6.1"

    implementation "androidx.room:room-runtime:$room_version"
    annotationProcessor "androidx.room:room-compiler:$room_version"

    implementation "androidx.lifecycle:lifecycle-viewmodel:2.8.7"
    implementation "androidx.lifecycle:lifecycle-livedata:2.8.7"
}
```

> **Copy the latest version numbers from the Android docs, then press Sync Now.**

PIECE 1

The Entity

```
@Entity(tableName = "contacts")
public class Contact {

    @PrimaryKey(autoGenerate = true)
    public int id;

    public String name;
    public String phone;
}
```

The contacts table

id	name	phone
1	Habib	017...
2	Faruk	018...
3	Mim	019...

PIECE 1

Entity Rules of Thumb

@Entity

This class is a table. Each field is a column

@PrimaryKey

Every row needs a unique key. autoGenerate counts for you

Public fields or getters

Room must be able to read and write every column

PIECE 2

The DAO

```
@Dao
public interface ContactDao {

    @Insert
    void insert(Contact contact);

    @Update
    void update(Contact contact);

    @Delete
    void delete(Contact contact);

    @Query("SELECT * FROM contacts ORDER BY name")
    LiveData<List<Contact>> getAll();
}
```

PIECE 2

The DAO Annotations

Annotation	What it does	You write
@Insert	Adds a row	Just the method
@Update	Changes a row by its primary key	Just the method
@Delete	Removes a row	Just the method
@Query	Runs your own SQL	The SQL string

PIECE 3

The Database

```
@Database(entities = {Contact.class}, version = 1)
public abstract class AppDatabase extends RoomDatabase {

    public abstract ContactDao contactDao();

    private static volatile AppDatabase INSTANCE;

    public static AppDatabase get(Context context) {
        if (INSTANCE == null) {
            synchronized (AppDatabase.class) {
                if (INSTANCE == null) {
                    INSTANCE = Room.databaseBuilder(
                        context.getApplicationContext(),
                        AppDatabase.class, "contacts.db").build();
                }
            }
        }
        return INSTANCE;
    }
}
```

WHY THAT ODD CODE?

One Database for the Whole App

Singleton

Opening a database is expensive, so build it once and reuse it

Application context

Never keep an Activity in a static field, because it leaks memory

version = 1

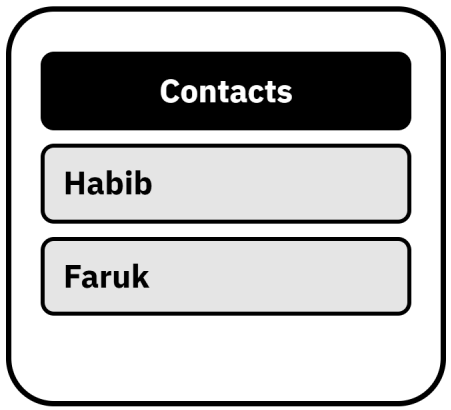
Change a table later and you must raise this number

A GOLDEN RULE

Never Query on the Main Thread



Slow query on the main thread: the screen freezes



Query in the background: the screen stays smooth

> Room throws an error if you try. Use a background thread or LiveData.

THE NICE PART

Queries That Update the Screen



> Return LiveData from the DAO and your list refreshes automatically on every insert, update or delete.

CHANGING YOUR MIND

Updating the Table Later

1. Add a new field to the Entity, such as email
2. Raise the database version from 1 to 2
3. Provide a Migration so old data is kept
4. In class, uninstalling the app is the quick fix

QUICK RECAP

Room

- Entity is the table, DAO is the list of actions, Database holds them
- Annotations replace most of the SQL plumbing
- Build the database once, never query on the main thread
- LiveData from the DAO updates the screen for free

> Next up: the Repository, one door in front of all your data.

TOPIC 5

The Repository

One door to all your data

The ViewModel does not need to know whether data came from a database or the internet. The Repository hides that.

AN ANALOGY

The Library Front Desk

You ask the librarian for a book. You do not care which shelf, which building or which warehouse it comes from. The Repository is that librarian for your app’s data.

One source of truth

The ViewModel asks one place for data

Hides the details

Database today, network tomorrow

Keeps rules together

Caching and syncing live in one class

WHERE IT SITS

ViewModel → Repository → Sources



REPOSITORY IN CODE

A Simple Repository

```
public class ContactRepository {

    private final ContactDao dao;
    private final ExecutorService io = Executors.newSingleThreadExecutor();

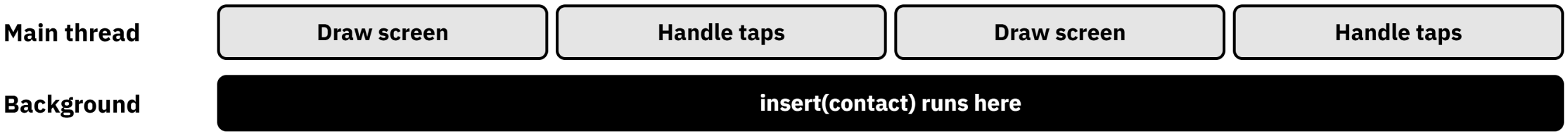
    public ContactRepository(Application app) {
        dao = AppDatabase.get(app).contactDao();
    }

    public LiveData<List<Contact>> getAllContacts() { return dao.getAll(); }

    public void insert(Contact c) { io.execute(() → dao.insert(c)); }
    public void delete(Contact c) { io.execute(() → dao.delete(c)); }
}
```

WHY EXECUTORSERVICE?

Keep the Main Thread Free



> The executor runs slow database work on another thread, so taps and scrolling never stall. Reads that return LiveData already run in the background.

SMART LOADING

Show Old Data, Fetch New Data

1. Show what is stored in Room straight away
2. Ask the web API for fresh data in the background
3. Save the fresh data into Room
4. The screen refreshes by itself because it observes Room

> This is caching. The user never stares at a blank screen, and we will use it again when we add networking.

QUICK RECAP

The Repository

- It is the single door between the ViewModel and every data source
- It hides whether data is local or remote
- Slow work runs on a background thread

> Next up: wire everything into a real Contact Manager.

TOPIC 6

The Contact Manager

Everything together

Layouts, ViewModel, Repository and Room, working as one team.

WHAT WE ARE BUILDING

The Contact Manager App



- A RecyclerView of contacts, sorted by name
- Add, edit and delete
- Everything is saved in Room
- The list updates itself, even after rotation

THE WHOLE PICTURE

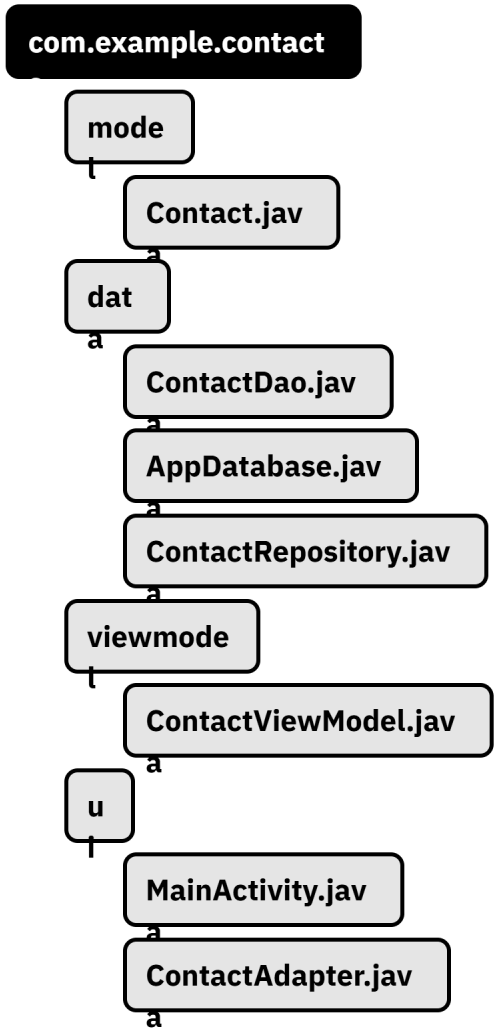
The Full MVVM Chain



> Data flows down on actions, and back up to the screen through LiveData.

KEEP IT ORGANISED

Project Structure



- One package per layer
- Open a package and you know what is inside
- Rule: ui never touches data directly

CONTACT MANAGER

The ViewModel

```
public class ContactViewModel extends AndroidViewModel {

    private final ContactRepository repository;
    private final LiveData<List<Contact>> contacts;

    public ContactViewModel(@NonNull Application app) {
        super(app);
        repository = new ContactRepository(app);
        contacts = repository.getAllContacts();
    }

    public LiveData<List<Contact>> getContacts() { return contacts; }
    public void insert(Contact c) { repository.insert(c); }
    public void delete(Contact c) { repository.delete(c); }
}
```

> **AndroidViewModel gives us an Application, which is what Room needs.**

CONTACT MANAGER

The Activity Observes

```
ContactViewModel vm =
        new ViewModelProvider(this).get(ContactViewModel.class);

ContactAdapter adapter = new ContactAdapter();
recyclerView.setLayoutManager(new LinearLayoutManager(this));
recyclerView.setAdapter(adapter);

vm.getContacts().observe(this, list → adapter.setContacts(list));

btnAdd.setOnClickListener(v → {
    Contact c = new Contact();
    c.name = etName.getText().toString();
    c.phone = etPhone.getText().toString();
    vm.insert(c);
});
```

CONTACT MANAGER

The Adapter Takes a List

```
public class ContactAdapter
        extends RecyclerView.Adapter<ContactAdapter.VH> {

    private List<Contact> contacts = new ArrayList<>();

    public void setContacts(List<Contact> list) {
        contacts = list;
        notifyDataSetChanged();
    }

    @Override public int getItemCount() { return contacts.size(); }

    // onCreateViewHolder and onBindViewHolder, as in the list lesson
}
```

> **The adapter just displays whatever list it is given. No database code in here.**

FOLLOW ONE CONTACT

What Happens When You Press Add

1. Activity builds a Contact and calls vm.insert()
2. ViewModel hands it to the Repository
3. Repository inserts it on a background thread
4. Room saves the row and emits a new list
5. LiveData tells the observer, the adapter updates, the new row appears

QUICK RECAP

The Contact Manager

- Each layer has one job and talks only to its neighbour
- The View observes, so it never needs a refresh button
- Room, Repository and ViewModel work as one chain

> Next up: a little polish. Shimmer loading.

TOPIC 7

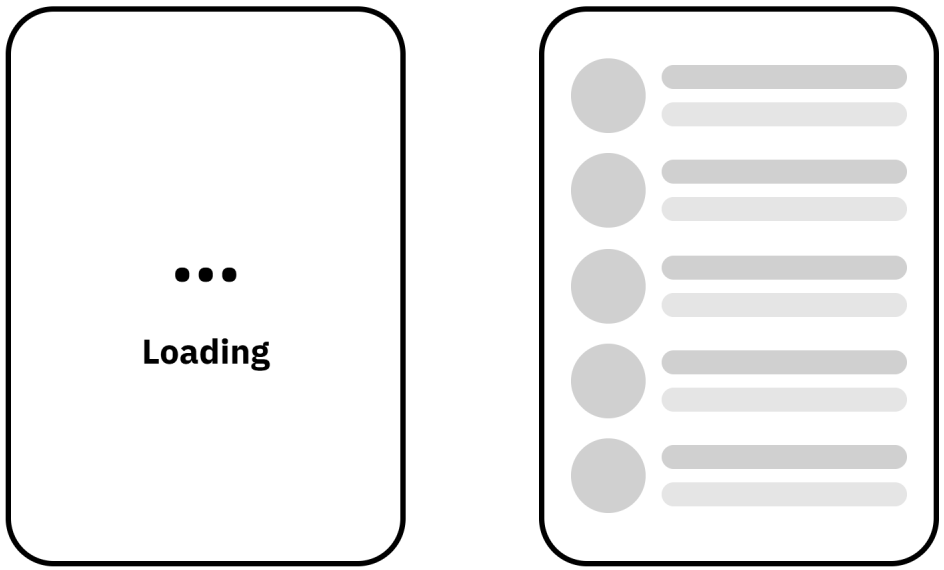
Shimmer Loading

Make waiting feel shorter

A screen that already looks like its content feels faster than a spinning circle.

WHY SHIMMER?

Blank Spinner vs. Skeleton



Spinner: nothing to look at

Skeleton: the layout appears at once

> A shimmering placeholder tells the user what is coming and makes the wait feel shorter.

SET UP

Add the Shimmer Library

```
dependencies {
    implementation 'com.facebook.shimmer:shimmer:0.5.0'
}
```

> It is Facebook’s open-source library. Press Sync Now after adding it.

THE LAYOUT

Wrap a Placeholder in ShimmerFrameLayout

```
<com.facebook.shimmer.ShimmerFrameLayout
    android:id="@+id/shimmerLayout"
    android:layout_width="match_parent"
    android:layout_height="wrap_content">

    <include layout="@layout/placeholder_contact_list" />

</com.facebook.shimmer.ShimmerFrameLayout>

<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/recyclerView" android:visibility="gone" ... />
```

> **The placeholder layout is a few grey rounded boxes in the shape of one list row, repeated.**

THE BEHAVIOUR

Start, Then Stop

```
shimmerLayout.startShimmer();

vm.getContacts().observe(this, list → {
    adapter.setContacts(list);

    shimmerLayout.stopShimmer();
    shimmerLayout.setVisibility(View.GONE);
    recyclerView.setVisibility(View.VISIBLE);
});
```

> **Room answers in a blink, so you will mostly see shimmer once networking joins the party.**

THE FULL STORY

From Skeleton to Real List



QUICK RECAP

Shimmer Loading

- Placeholders feel faster than spinners
- ShimmerFrameLayout wraps a skeleton layout
- Start it on open, stop it when data arrives

> Next up: a quick quiz, then the common mistakes.

TEST YOURSELF

Which Layer?

1. The click listener on the Add button
2. A @Query that selects all contacts
3. Checking that the name is not empty
4. Exposing LiveData<List<Contact>> to the screen
5. Deciding whether to use cached data

> Options: View, ViewModel, Repository or Data source (DAO)

THE ANSWERS

Which Layer? Solved

Code	Layer
Click listener on Add	View (Activity)
@Query selecting contacts	Data source (DAO)
Name is not empty	ViewModel
Exposing LiveData	ViewModel
Cache or fresh data?	Repository

REMEMBER THE POLL?

Where does the “password has six characters” rule belong?

Best answer: C

In its own logic class: the Presenter in MVP or the ViewModel in MVVM

Why not the Activity?

It only draws the screen and forwards taps, so rules stay out of it

LEARN FROM THESE NOW

Common Mistakes

Fat ViewModel

Do not put views or Context in it

Querying on the main thread

Room will refuse. Use the Repository

Forgetting notifyDataSetChanged

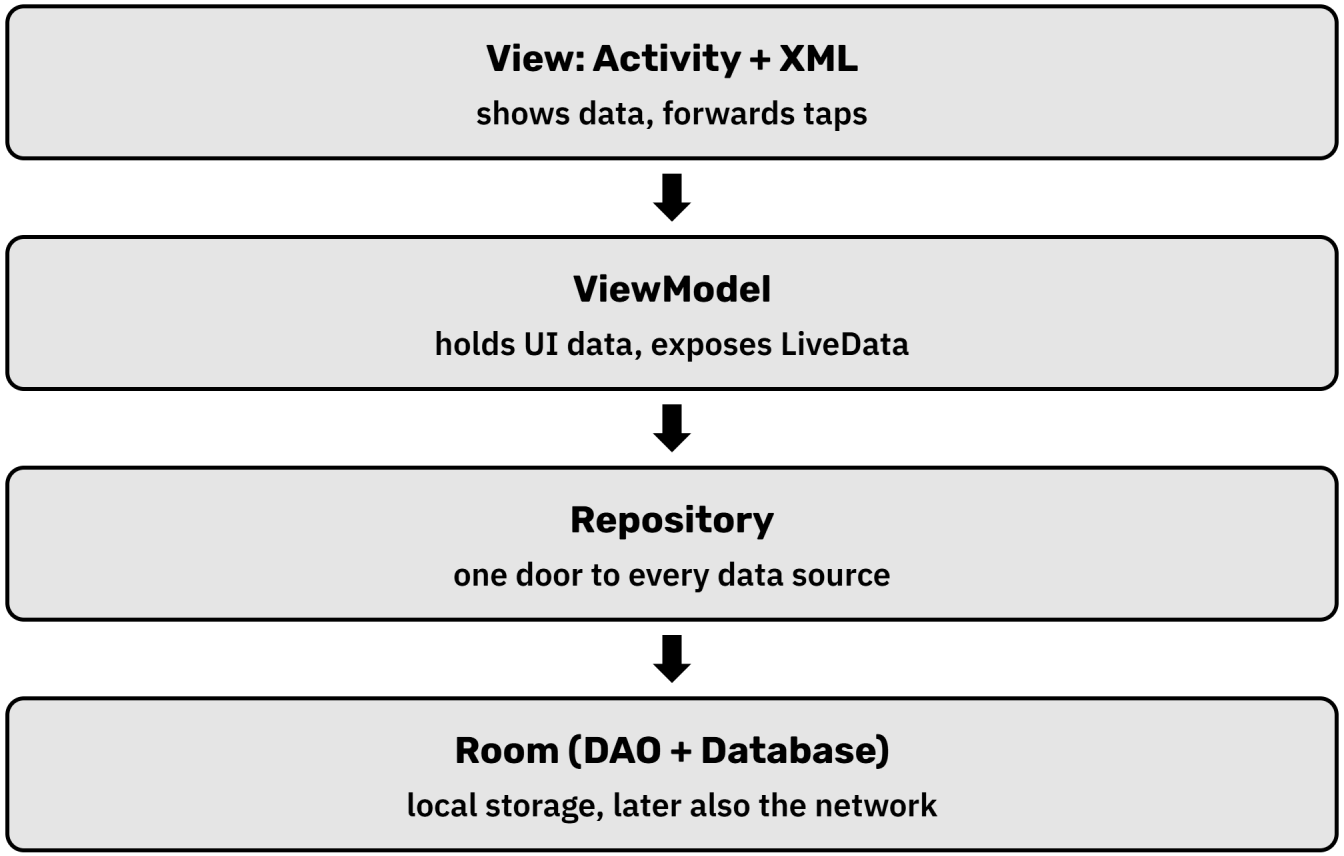
The list updates but the screen does not

Skipping the version bump

Changing a table without raising version crashes the app

ONE SLIDE TO REMEMBER

The Whole Architecture



KEEP THIS ONE

Where Does This Code Go?

If it is about	Put it in
Showing things, handling taps	Activity / Fragment
Screen data and screen rules	ViewModel
Where data comes from	Repository
Tables and queries	Entity, DAO, Database
Loading placeholders	View layer, with Shimmer

Questions?

MVC MVP MVVM Room Repository

Ekramul Haque Amin

Lecturer, AI & Data Science · Mobile Application and Development (0613-409)